DATA STRUCTURES × ALGORITHMS

SMART RESUME SCREENING TOOL



GROUP NO:4

Group Members:
2510080029-Sudeeptha
2510080037-Lokrey Latika
2510080043-Sai Siri
2510080061-Kovidha

Under the guidance of
Dr.Anitha Mam

Problem Background

Resumes arrive at scale — in varied formats, with scattered evidence and inconsistent skill labels.

PROBLEM STATEMENT

Develop a DSA-based system that analyzes, matches, scores and ranks candidates efficiently — beyond slow manual review and brittle exact-keyword filters.

Literature Review

01

RESUME PARSING

Skills, education, experience and certifications become structured data.

02

NLP SCREENING

Semantic context reveals meaning beyond literal words.

03

MACHINE LEARNING

Patterns predict classification and candidate suitability.

04

SEMANTIC MATCHING

Similarity aligns resumes with job descriptions.

COMMON THREAD / parsing → skill extraction → similarity → ranking

Comparision of existing approaches

COMPARISON OF EXISTING APPROACHES

APPROACH	METHODS	STRENGTHS	LIMITATIONS	SUITABILITY
MANUAL COMPARISON	• Human reading and comparison	• Understands context and meaning • Flexible	• Very time-consuming • Inconsistent results • Not scalable	★★☆☆☆ Suitable only for small documents
KEYWORD MATCHING	• Bag of Words • TF-IDF • N-gram Matching	• Simple to implement • Fast for large documents	• Fails on paraphrased or modified text • Only exact word match	★★★☆☆ Works for basic similarity detection
MACHINE LEARNING APPROACHES	• SVM, Naive Bayes • Feature Engineering • Classification Models	• Detects various plagiarism types • Learns from data	• Needs labeled datasets • Feature engineering is complex • Training cost	★★★☆☆ Good with proper training data
NLP / DEEP LEARNING APPROACHES	• Word Embeddings • Transformers • Semantic Similarity	• Captures semantic meaning • Handles paraphrasing better	• High computational cost • Requires large datasets • Complex implementation	★★★☆☆ Effective but resource intensive
PROPOSED DSA-BASED APPROACH	• KMP, Z-Algorithm • Rabin-Karp • Edit Distance • Suffix Array + LCP	• Efficient exact & fuzzy matching • No training data needed • Lower computational cost	• Mainly text-based • May miss deep semantic similarity	★★★★☆ Efficient for exact & structural similarity

Research Gap

01 — LARGE RESUME VOLUMES

Recruiters need to process many applications efficiently .

02 — LIMITATION OF EXACT KEYWORD MATCHING

A candidate may have the required skill but represent it differently.

Example:

Java Script ↔ JavaScript

03 — EFFICIENT TEXT SEARCH

Searching large resume documents repeatedly can increase processing time.

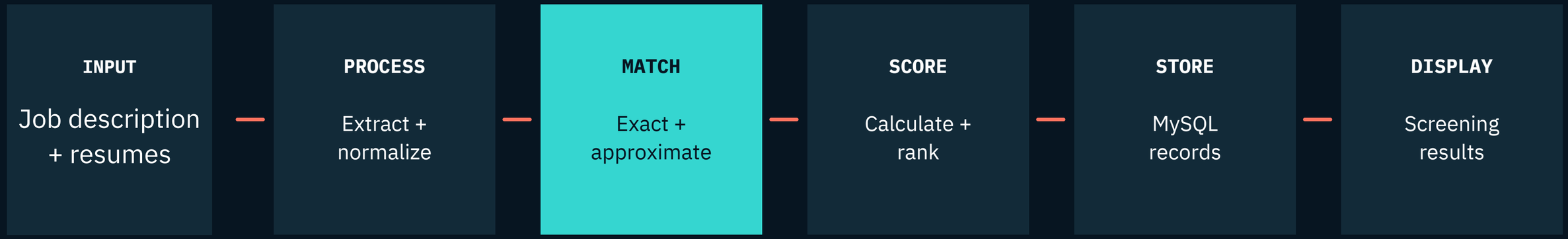
04 — APPROXIMATE SKILL MATCHING

Similar terms should be detected instead of depending only on exact matches.

RESEARCH GAP

There is a need for an efficient screening approach that combines exact string matching, approximate matching, and candidate ranking using DSA techniques.

Proposed Methodology



CORE / Resume Processing → Skill Matching → Similarity Analysis → Scoring → Ranking

Resume Text Preprocessing

01 CLEAN

Remove punctuation, unwanted characters and excess spaces.

02 NORMALIZE

Standardize case and text representation.

03 TOKENIZE

Split content into searchable words and terms.

BEFORE “Experienced Software Developer with JAVA, MySQL & JavaScript skills.”

AFTER experienced software developer java mysql javascript skills

DSA-Based Skill Matching

EXACT

KMP

LPS array enables efficient scanning.

Java

MULTI-PATTERN

RABIN–KARP

Rolling hashes search many skills.

Java | Python | MySQL

APPROXIMATE

EDIT DISTANCE

Insert, delete, substitute.

JavaScript ⇔ Javascript

EXACT KMP + RABIN–KARP / APPROXIMATE EDIT DISTANCE

Candidate Scoring & Ranking

SKILL MATCH	EXPERIENCE	EDUCATION	SIMILARITY

A	5/5 skills • experience + education match	92% HIGH PRIORITY
B	3/5 skills • partial experience match	67% MEDIUM PRIORITY

DataBase & Result Storage



DataBase Fields

Candidate_ID	Candidate_Name	Skills
Education	Experience	Match_Score
Ranking	Status	Screening Date

Implementation Plan

01 Literature Review

02 System & Methodology Design

03 Resume + Job Data Collection

04 Text Extraction + Preprocessing

05 Hashmap Skill Storage

06 KMP Implementation

07 Rabin–Karp Implementation

08 Edit Distance Implementation

09 Match Score + Ranking

10 MySQL Integration

11 Testing + Final Demo

Thank You!